Scikit-learn Classification

Introduction to Scikit-learn

- Scikit-learn
  - Machine learning library built on **NumPy**, **SciPy** and **Matplotlib**
- What Scikit-learn can do
  - **Supervised** learning
    - Regression, classification
  - **Unsupervised** learning
    - Clustering
  - Data **preprocessing**
    - Feature extraction, feature selection, dimensionality reduction

Introduction to Scikit-learn

- What Scikit-learn **cannot** do
  - Distributed computation on multiple computers
    - Only multi-core optimization
  - Deep learning
    - Use Keras and Tensorflow instead

Introduction to Scikit-learn

- Scikit learn models work with structured data
  - Data must be in the form of **2D Numpy arrays**
    - Rows represent the **samples**
    - Columns represent the **attributes (or features)**
  - This table is called **features matrix**

shape = (3, 3)

	Price	Quantity	Liters
Sample 1	1.0	5	1.5
Sample 2	1.4	10	0.3
Sample 3	5.0	8	1

Introduction to Scikit-learn

- Features can be
  - **Real** values
  - **Integer** values to represent categorical data
- If you have strings in your data, you first have to convert them to integers (**preprocessing**)

Input data

1.0	**January**	1.5
1.4	**February**	0.3
5.0	**March**	1

→

Features matrix

1.0	**0**	1.5
1.4	**1**	0.3
5.0	**2**	1

Introduction to Scikit-learn

- Also **missing values** must be solved before applying any model
  - With imputation or by removing rows/columns

Input data

1.0	0.5	1.5
1.4	**NaN**	0.3
5.0	0.5	1

→

Features matrix

1.0	0.5	1.5
1.4	**0.5**	0.3
5.0	0.5	1

Input data

1.0	0.5	1.5
1.4	**NaN**	0.3
5.0	0.5	1

→

Features matrix

1.0	0.5	1.5
5.0	0.5	1

Introduction to Scikit-learn

- For **unsupervised** learning you only need the features matrix
- For **supervised** learning you also need a **target** array to train the model
  - It is typically one-dimensional, with length n_samples
    - May be 2-dimensional for multi-output models
      - Shape: (n_samples, n_classes)

Features matrix
shape = (n_samples, n_features)

1.0	5	1.5
1.4	10	0.3
5.0	8	1

Target array
shape = (n_samples,)

A
A
B

Introduction to Scikit-learn

- The target array can contain
  - Integer values, each corresponding to a class label

Target labels

Dog
Dog
Cat

→

Target array

0
0
1

  - Real values for regression

Target array

0.4
1.8
-6.9

Introduction to Scikit-learn

- Scikit-learn estimator API
  - All models are represented with Python classes
  - Their classes include
    - The values of the **hyperparameters** used to configure the model
    - The values of the **parameters** learned after training
      - By convention, the attribute names end with an underscore
      - E.g., feature_importances_, tree_
    - The **methods** to train the model and make inference
  - Scikit-learn models are provided with sensible **defaults** for the hyperparameters

Introduction to Scikit-learn

- Scikit learn models follow a simple, shared **pattern**

1. **Import** the model that you need to use
   - `from sklearn.[…] import ModelName`
2. **Build** the model, setting its hyperparameters
   - `model = ModelName(hyperparam1=value1, … )`
3. **Train** model parameters on your data
   - `model.fit(X_train, y_train)`

- Use the model to make predictions
  - `model.predict()` or `model.transform()`

- Sometimes fit and predict/transform are implemented within the same class method

Predictors vs transformers

- In scikit-learn, we separate **estimators** into:
- Predictors
  - An estimator supporting predict() and/or fit_predict()
  - Used to predict values, generally after a training (fitting) step
  - Includes classifiers, regressors, outlier detectors, clusterers
- Transformers
  - An estimator that supports transform() and/or fit_transform()
  - Used to compute a new representation of the same data
  - E.g., Min-Max scaling, PCA, Feature discretizers, Tf-idf

Methods for predictors

- **fit():** learn model parameters from input data
  - E.g. train a classifier on a training set
- **predict():** apply model parameters to make predictions on data
  - E.g. predict class labels for new samples
- **fit_predict():** fit model and make predictions
  - E.g. apply clustering to data

Methods for transformers

- **fit():** learn model parameters from input data
  - E.g. learn minimum value and maximum value for each feature, in Min-Max scaler
- **transform()**: transform data into a different representation
  - E.g. rescale input data to the [0, 1] range
- **fit_transform():** fit model and transform data
  - E.g. apply PCA to transform data

Classification

- Classification:
  - Given a 2D features matrix X
    - X.shape = (n_samples, n_features)
  - The task consists of assigning a class label y_pred to each data sample
    - y_pred.shape = (n_samples)

1.0	5	1.5
1.4	10	0.3
…	…	…

X

A
B
B

y_pred

Classification

By following the estimator API pattern:

- Import a model

```
from sklearn.tree import DecisionTreeClassifier
```

- Build model object

```
clf = DecisionTreeClassifier()
```

Classification

- Important decision tree hyperparameters:

```
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier(max_depth = 10,
                             min_impurity_decrease=0.01)
```

- Hyperparameters:
  - *max_depth*: maximum tree height
    - Default = None
  - *min_impurity_decrease*: split nodes only if impurity decrease above threshold
    - Default = 0.0

Classification

- Train model with ground-truth labels

```
In [1]: clf.fit(X_train, y_train)
```

  - This operation builds the decision tree structure
    - X_train is the 2D Numpy array with input features (**features matrix**)
    - y_train is a 1D array with ground-truth labels

6.1	3.1	2
1.8	12	0.15
…	…	…

X_train

0
2
1

y_train

Classification

- Predict class labels for new data

```
In [1]: y_pred = clf.predict(X_test)
```

```
Out[1]: [3, 1, 1, 1, 2, 2, 0]
```

- This operation shows the capability of classifiers to make predictions for unseen data

1.0	5	1.5
1.4	10	0.3
…	…	…

X_test

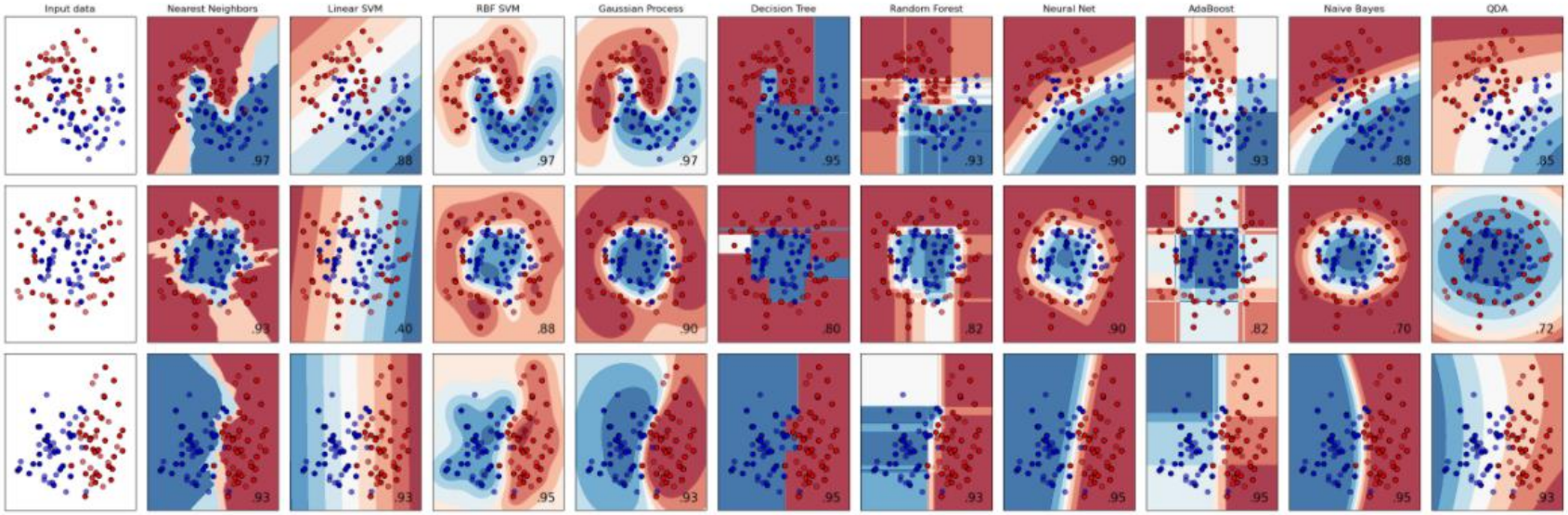
→

1
3
3

y_pred

Classification

- Take a look at all the other models in the scikit-learn documentation
  - https://scikit-learn.org/stable/auto_examples/classification/plot_classifier_comparison.html



Classification

- To **choose** the most appropriate machine learning model for your data you have to **evaluate** its performance

- Evaluation can be performed according to a **metric (scoring function)**
  - E.g. accuracy, precision, recall

Validation techniques

- Hold-out
  - Set aside part of the dataset as a “test set”
  - Use the rest for training

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Classification

- Evaluation = compare the following two vectors
  - y_true (y): the expected result (**ground truth**)
  - y_pred $(\hat{y})$: the prediction made by your model

- Main evaluation metrics for classification:
  - Accuracy: % of correct samples
  - Precision(c): % of correct samples among those predicted with class c
  - Recall(c); % of correct samples among those that belong to class c in ground truth
  - F_1 score(c): harmonic mean between precision and recall

Classification

- Evaluation metrics with Scikit-learn
  - `accuracy_score()`, `precision_score()`, `recall_score()`, `f1_score()`, …

- For binary problems:
  - precision/recall/F1 are computed for the <u>positive</u> class
    - can be specified with `pos_label` (default `1`)

```
from sklearn.metrics import accuracy_score, precision_score, f1_score,
recall_score

y_true = [0, 0, 1, 0, 1, 1, 0]
y_pred = [0, 1, 1, 1, 0, 1, 1]

acc = accuracy_score(y_true, y_pred)

prec = precision_score(y_true, y_pred) # precision for the positive (1) class

f1 = f1_score(y_true, y_pred, pos_label=0) # f1 score for the negative (0) class
```

Aggregate metrics

- For multi-class problems, we can choose how to aggregate the scores on the different classes
- `average=None`
  - Does not aggregate
  - Returns an array with the metric for all classes
- `average="macro"`
  - Computes the unweighted mean across all classes
  - Same as (for f1): `f1_score(y_true, y_pred, average=None).mean()`
- `average="weighted"`
  - Weighs each class's score by the corresponding support (number of instances in ground truth values)
  - Same as (for f1): `(f1_score(y_true, y_pred, average=None) * np.unique(y_true, return_counts=True)[1] / len(y_true)).sum()`

Classification

- **Confusion matrix**
  - Useful tool when inspecting the classification results in more detail

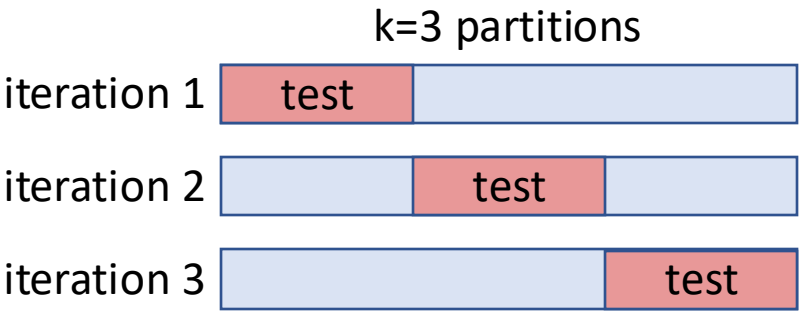
In [1]:
```
from sklearn.metrics import confusion_matrix


conf_mat = confusion_matrix(y_test, y_test_pred)
print(conf_mat)
```

Out[1]:

		predicted		
		0	1	2
actual	0	[[45,	0,	1],
	1	[0,	43,	0],
	2	[0,	3,	42]]

K-fold cross-validation

- Split dataset into k folds
- Use one fold for testing, the other for training
- Repeat k times




```python
from sklearn.model_selection import KFold

# K-Fold with 3 splits
kfold = KFold(n_splits=3, shuffle=True)

for train_indices, test_indices in kfold.split(X, y):
        ... executed 3 times, 1 for each k-fold iteration ...
        X_train, y_train = X[train_indices], y[train_indices]
        X_test, y_test = X[test_indices], y[test_indices]
```

`kfold.split()` returns at each **iteration** a tuple with two **arrays**:
- `train_indices`: array of the **indices** (row number) of the training sample
- `test_indices`: array of the indices of the test samples

Cross-validation

- Method 2: use cross_val_score()

```
from sklearn.model_selection import cross_val_score


clf = DecisionTreeClassifier()
acc = cross_val_score(clf, X, y, cv=5, scoring='accuracy')
```

- Parameters:
  - clf = the model that you want to be trained
  - X, y = your dataset, where cross-validation will be performed
- Important: this method **does not shuffle** data
  - Manually shuffle them when necessary (suggested)

Cross-validation

- Method 2: use cross_val_score()

```
from sklearn.model_selection import cross_val_score


clf = DecisionTreeClassifier()
acc = cross_val_score(clf, X, y, cv=5, scoring='accuracy')
```

- Parameters:
  - cv = number of partitions for cross-validation
  - scoring = scoring function for the evaluation
    - E.g. ‘f1_macro’, 'f1_micro', ‘accuracy’, 'precision_macro'

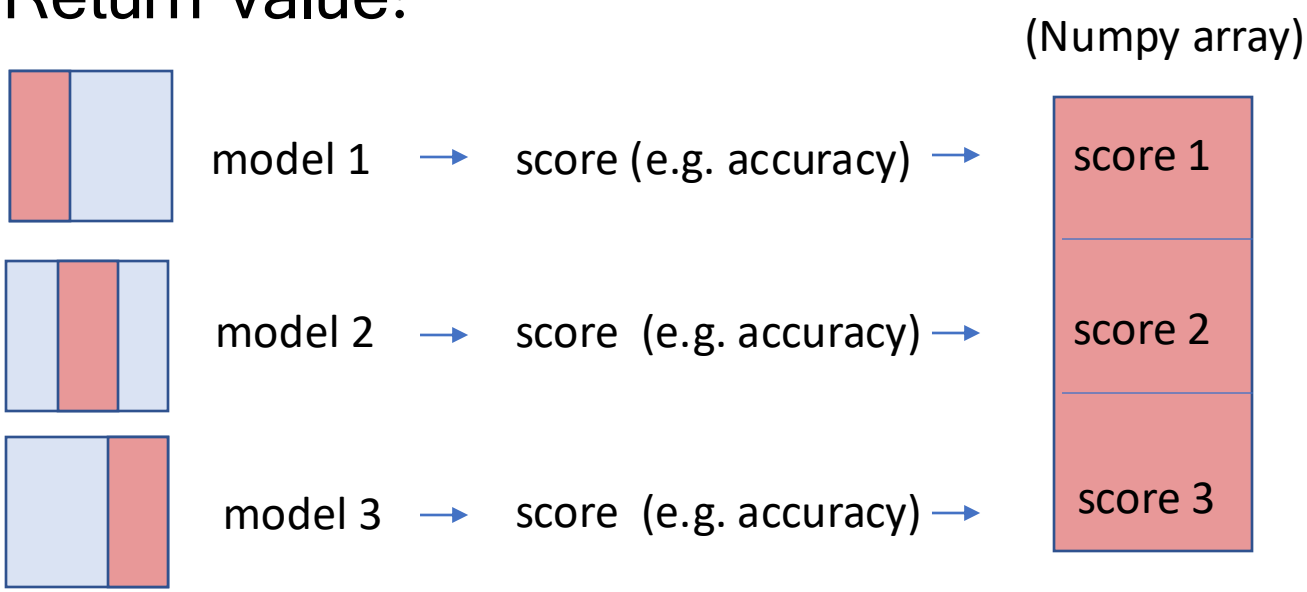
Cross-validation

- Method 2: use cross_val_score()

In [1]:
```
cross_val_score(clf, X, y, cv=3, scoring='accuracy')
```

Out[1]:
```
array([0.85, 0.86, 0.833])
```

- Return value:



Cross-validation

- Method 3: use cross_val_predict()

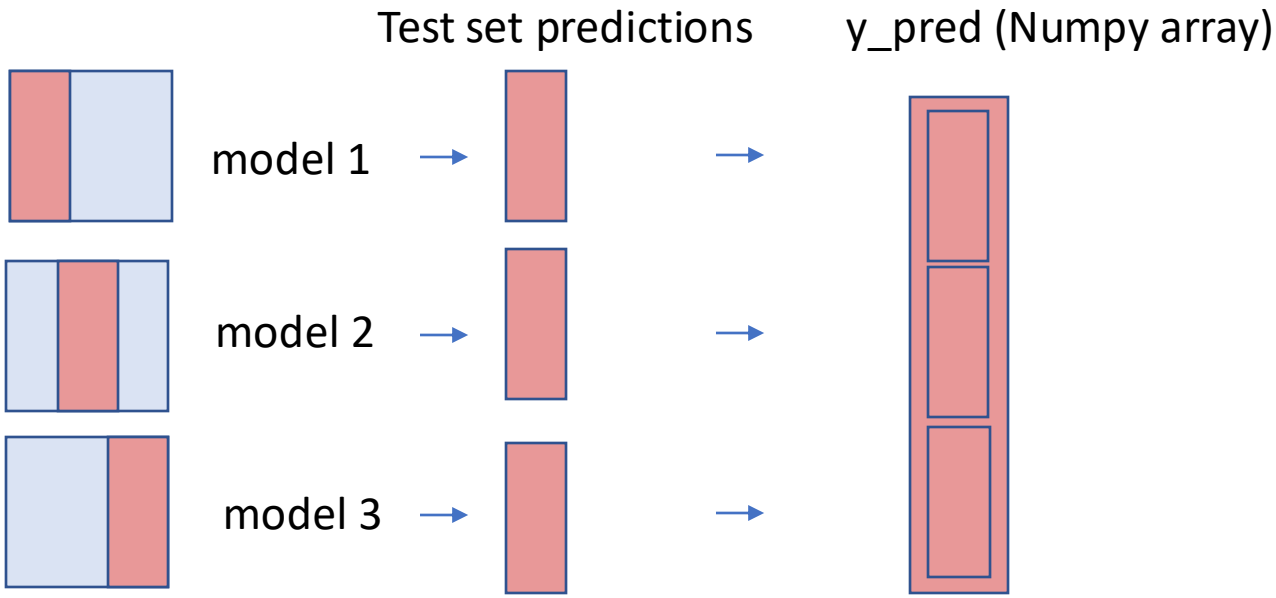
```
from sklearn.model_selection import cross_val_predict
y_pred = cross_val_predict(clf, X, y, cv=3)
```

- This method returns a Numpy array with the predictions of the *cv* models trained during cross validation
- Data is **not shuffled**

Cross-validation

- Method 3: use cross_val_predict()

```
from sklearn.model_selection import cross_val_predict
y_pred = cross_val_predict(clf, X, y, cv=3)
```



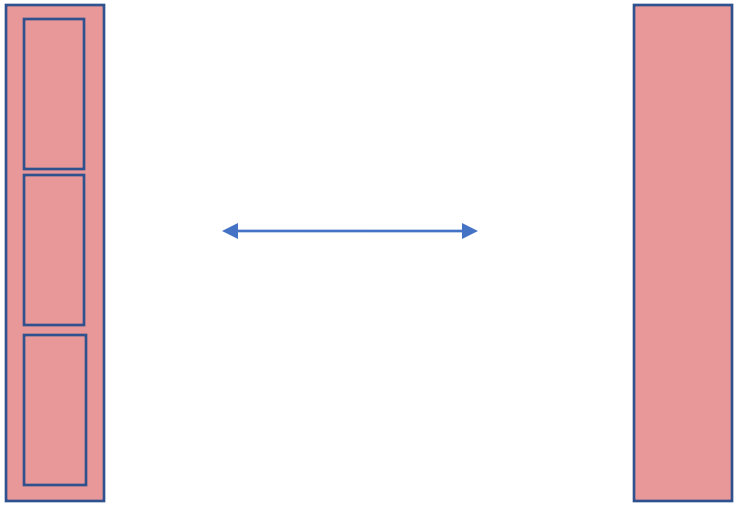
Cross-validation

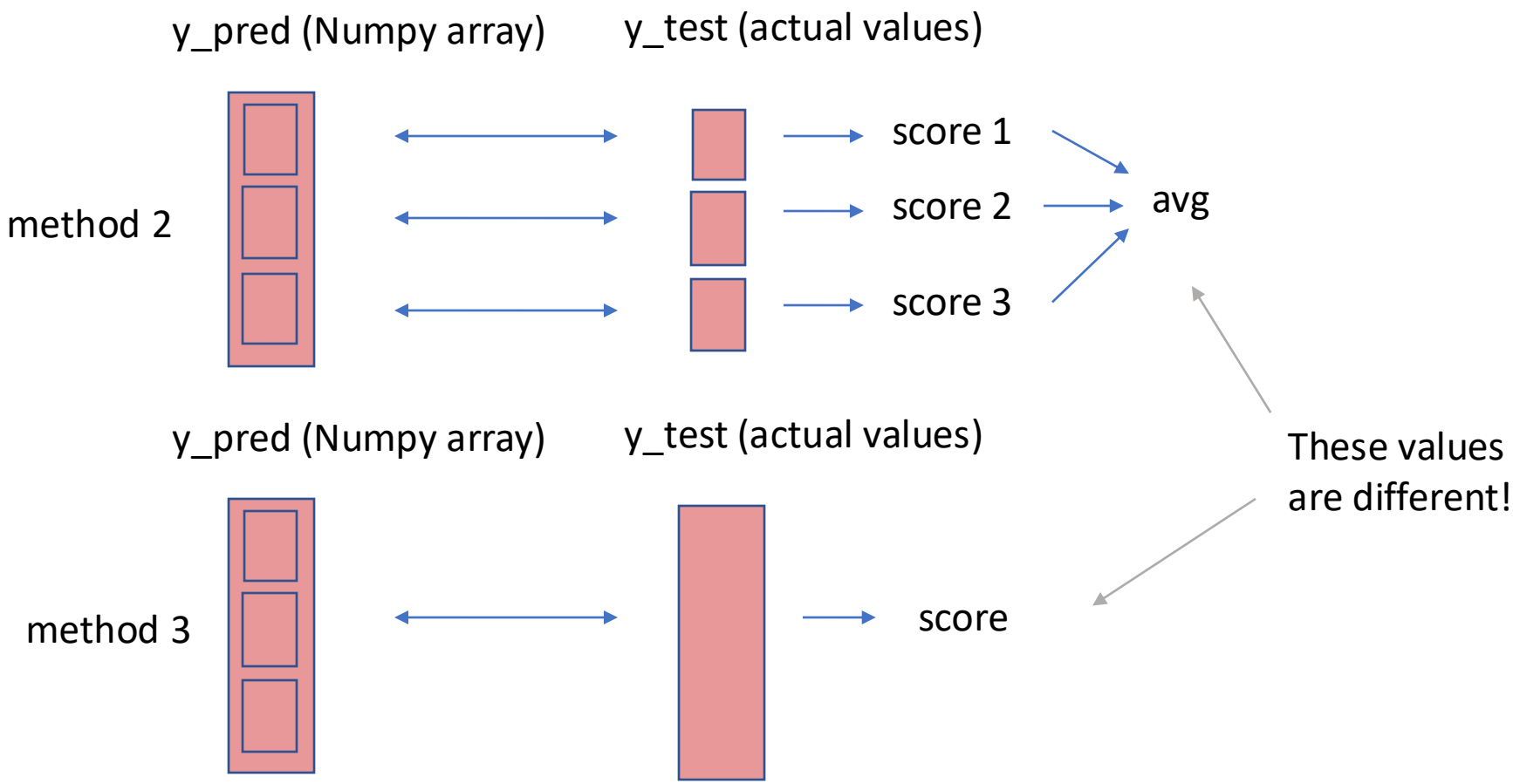
- Method 3: use cross_val_predict()
  - Finally you can evaluate the predictions

```
acc = accuracy_score(y_test, y_test_pred)
```

y_pred (Numpy array)

y_test (actual values)



Cross-validation

- Difference between method 2 and method 3